

sea!me

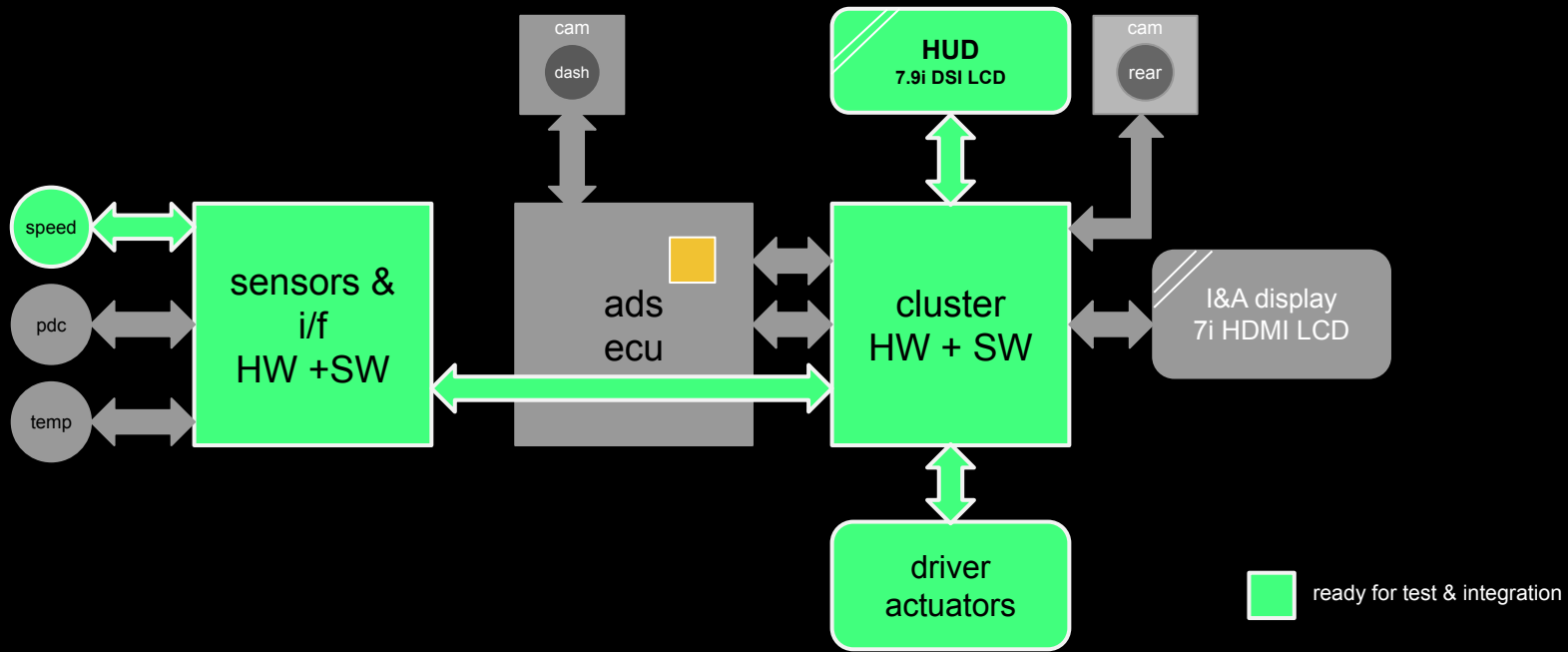
software engineering in automotive
and mobility ecosystems

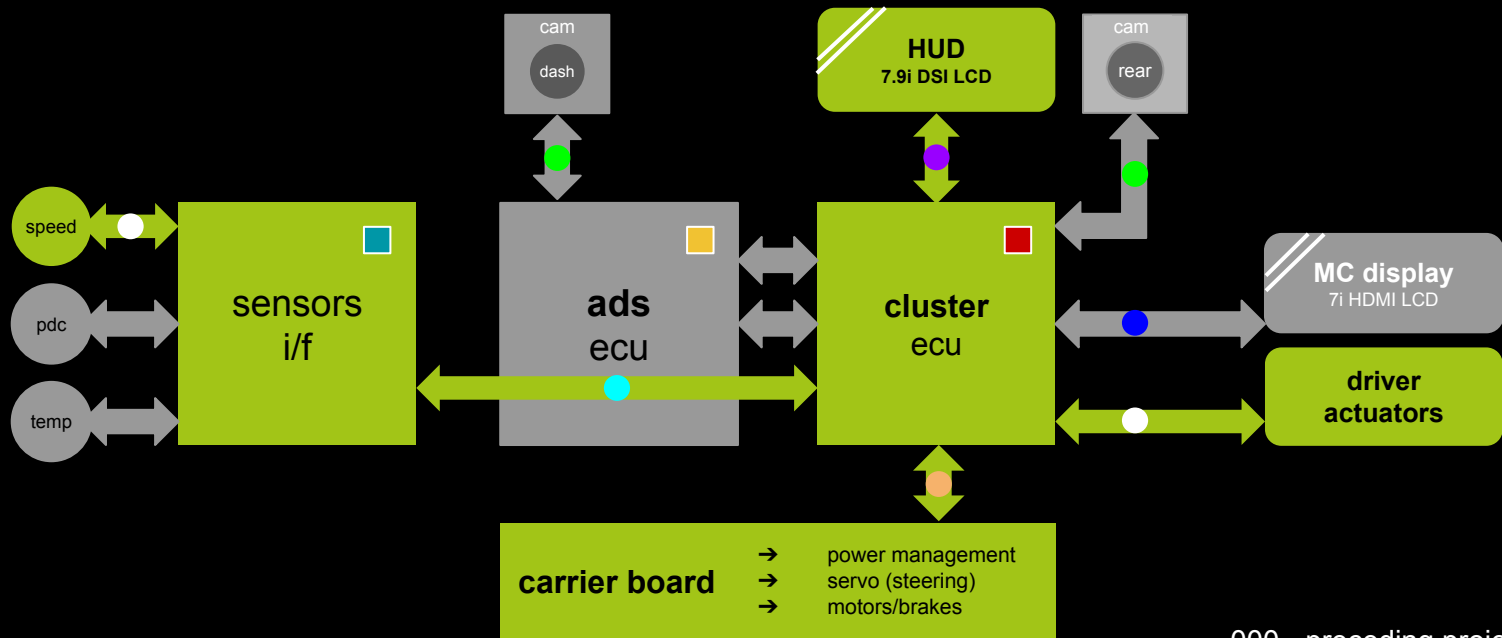
team 07

ADS : lane detection

project planning

ADR HW : validation





003 - comm protocols

- white circle: custom
- orange circle: spi/i2c
- cyan circle: can
- tcp/ip

004 - video protocols

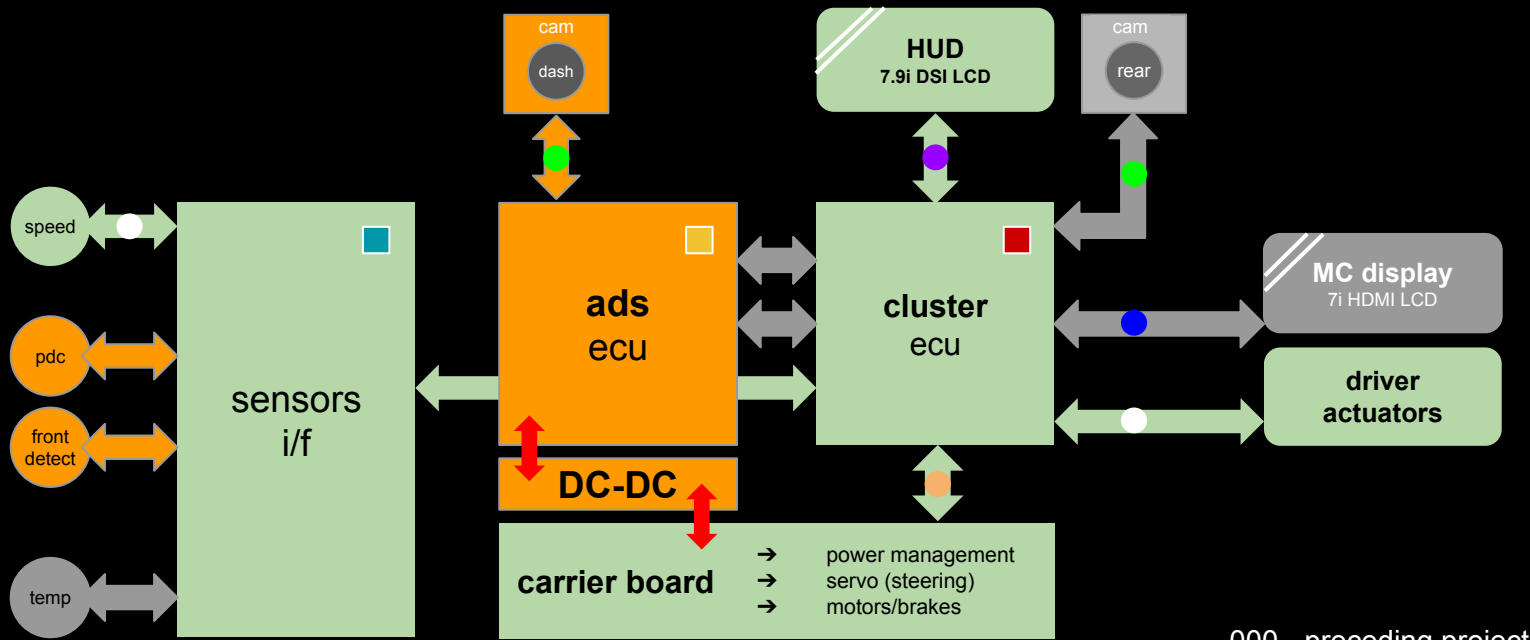
- purple circle: dsi
- blue circle: hdmi
- green circle: csi

001 - control units

- blue square: Arduino UNO R3
- orange square: Jetson Nano OKdo
- red square: Raspberry Pi 4B

000 - preceding project

green square: ready



003 - comm protocols

- white circle: custom
- orange circle: spi/i2c
- orange circle: tcp/ip
- cyan circle: can

004 - video protocols

- purple dot: dsi
- green dot: cs
- blue dot: hdmi

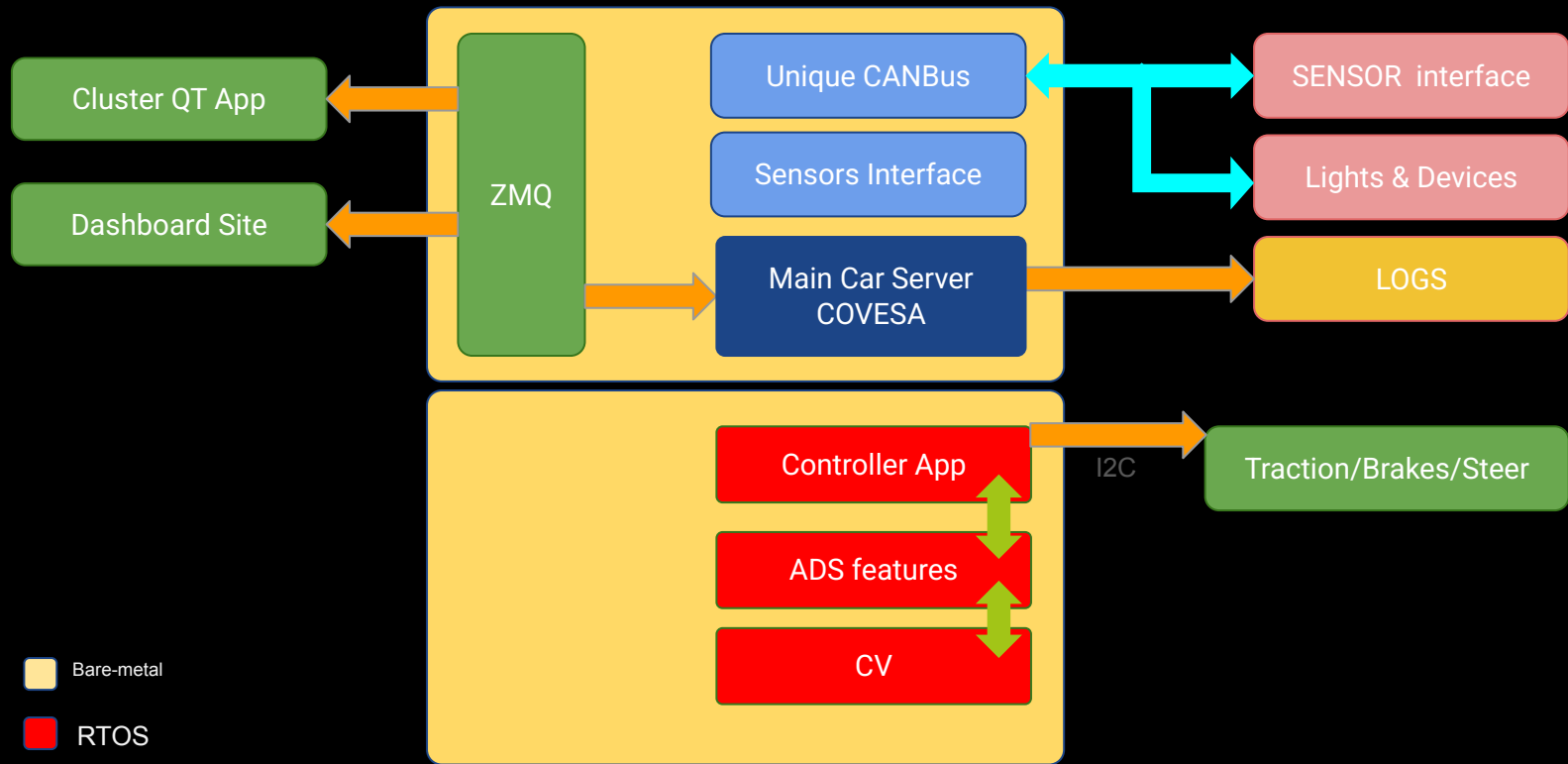
001 - control units

- blue square: Arduino UNO R3
- yellow square: Jetson Nano OKdo
- red square: Raspberry Pi 4B

000 - preceding project

- green square: ready
- orange square: ADS HW

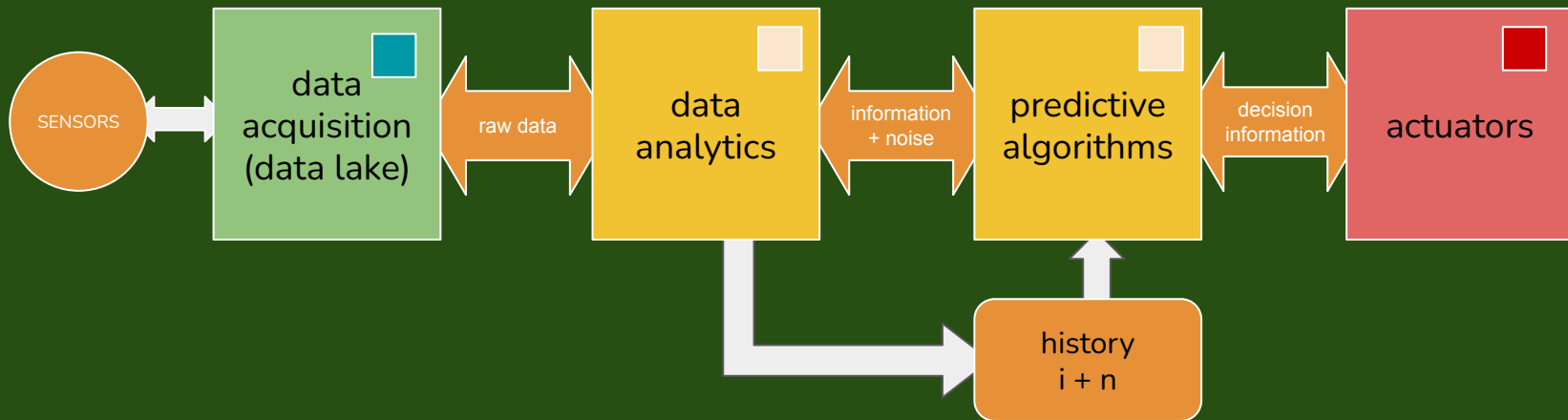
ADR SW : decision

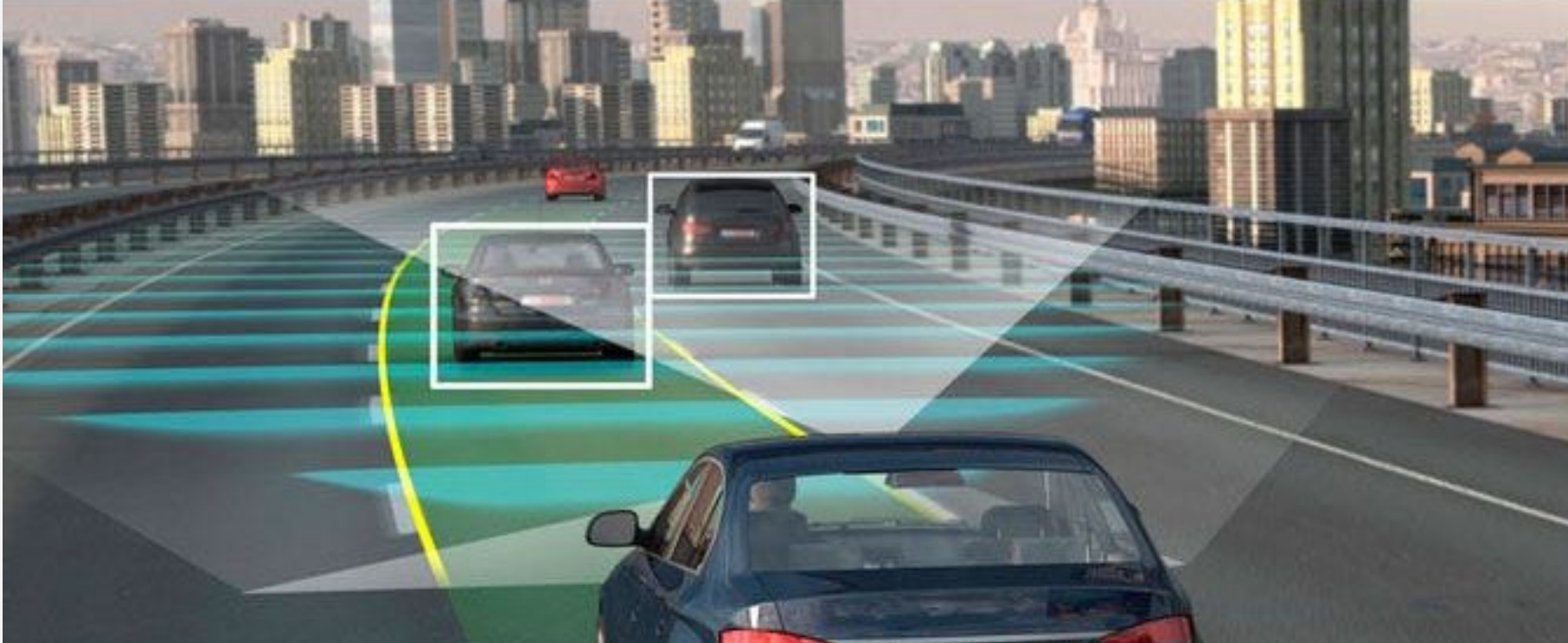


lane detection

SPRINT 1 : retrospective

we did learn about





lane detection

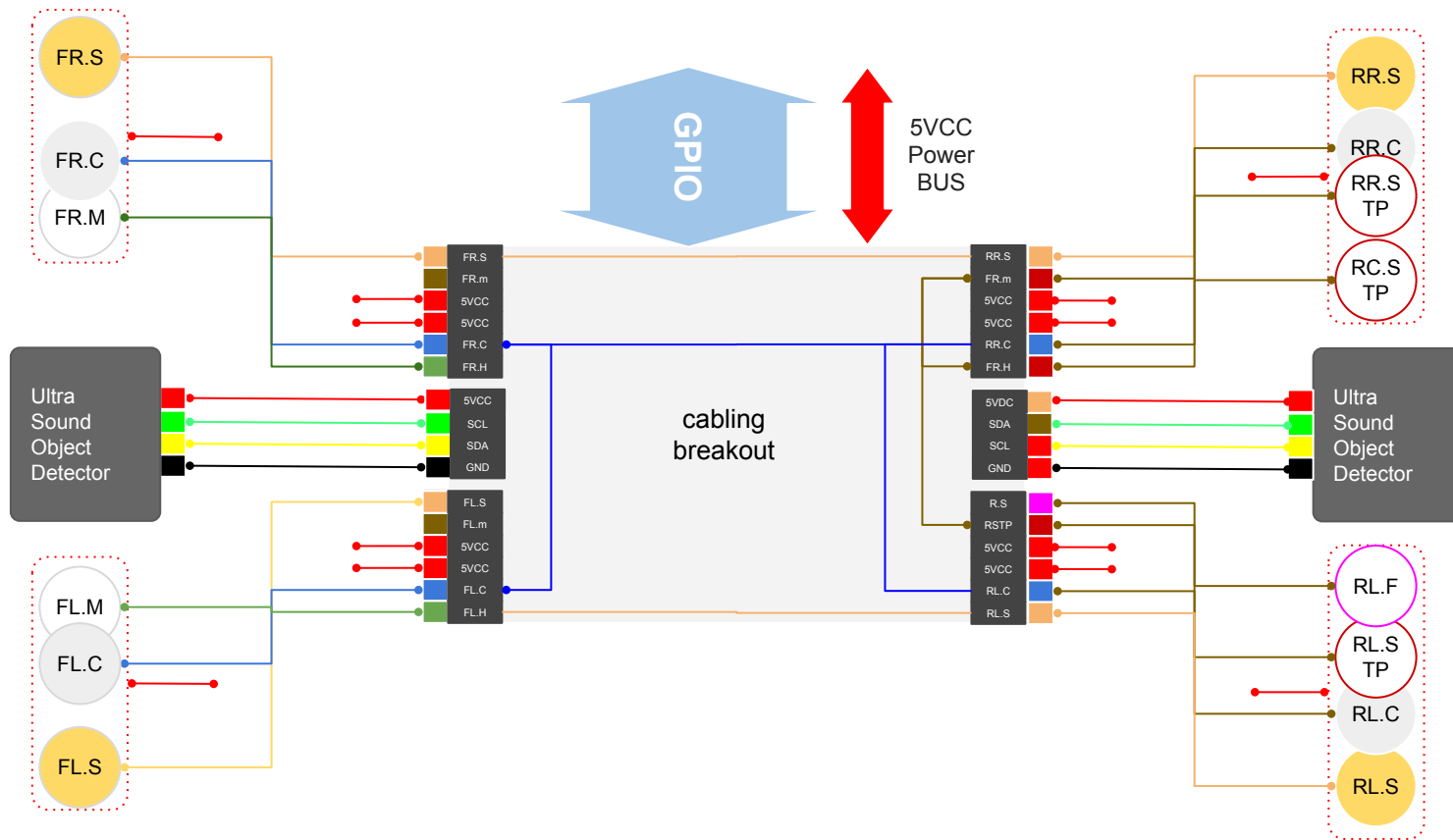
SPRINT 2 : RETROSPECTIVE

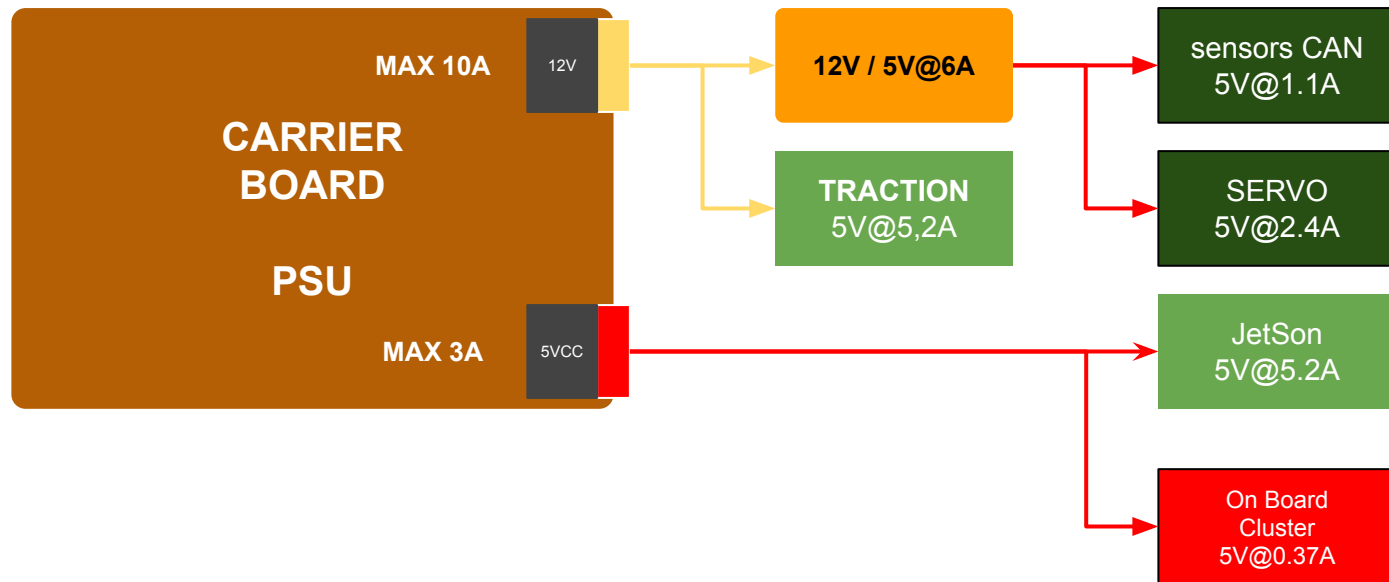
energy planning / management

energy planning / management

FRONT

REAR



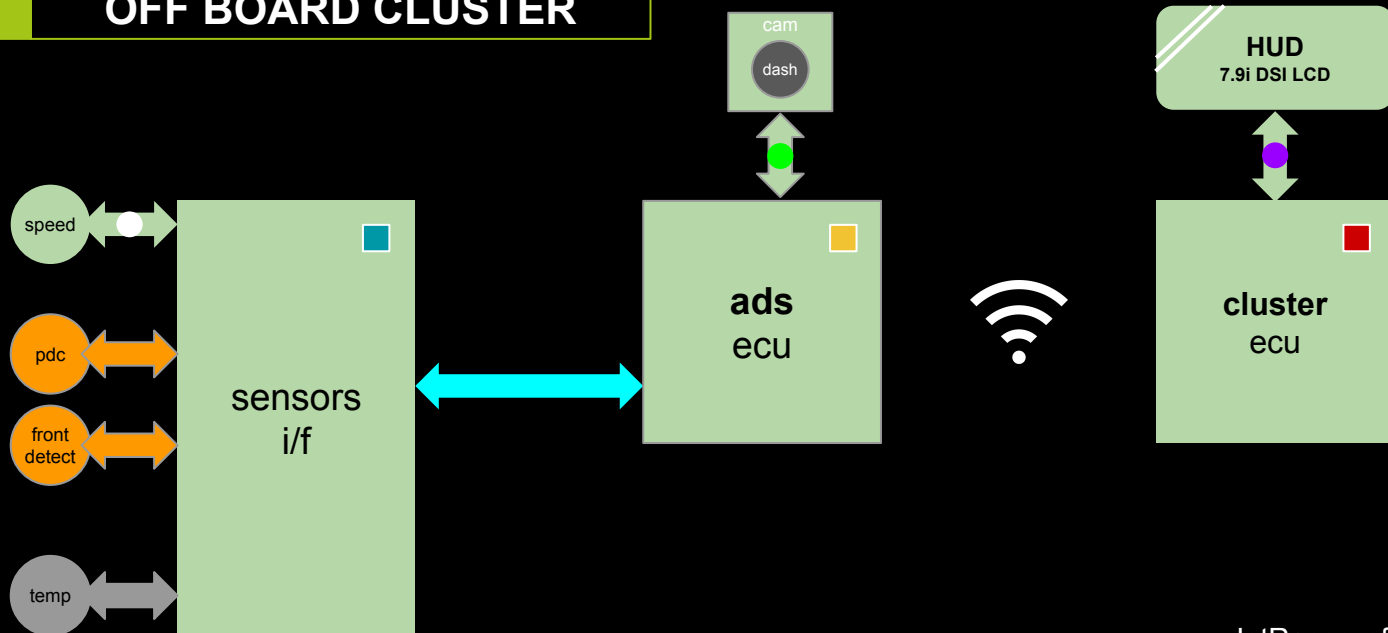


jetson nano setup

jetson nano setup

issue :

OFF BOARD CLUSTER



003 - comm protocols

- white circle: custom
- orange circle: spi/i2c
- grey circle: tcp/ip
- blue circle: can

004 - video protocols

- purple circle: dsi
- green circle: hdmi
- blue circle: csi

001 - control units

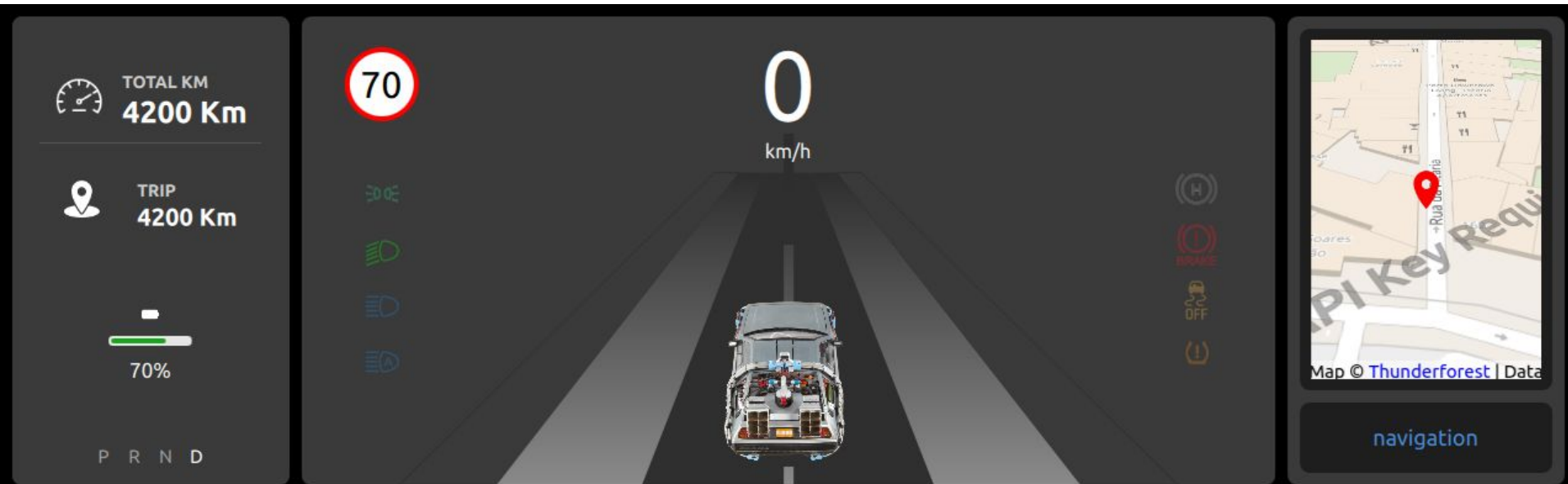
- blue square: Arduino UNO R3
- orange square: Jetson Nano OKdo
- red square: Raspberry Pi 4B

JetRacer refactoring

- green square: ready
- orange square: in progress

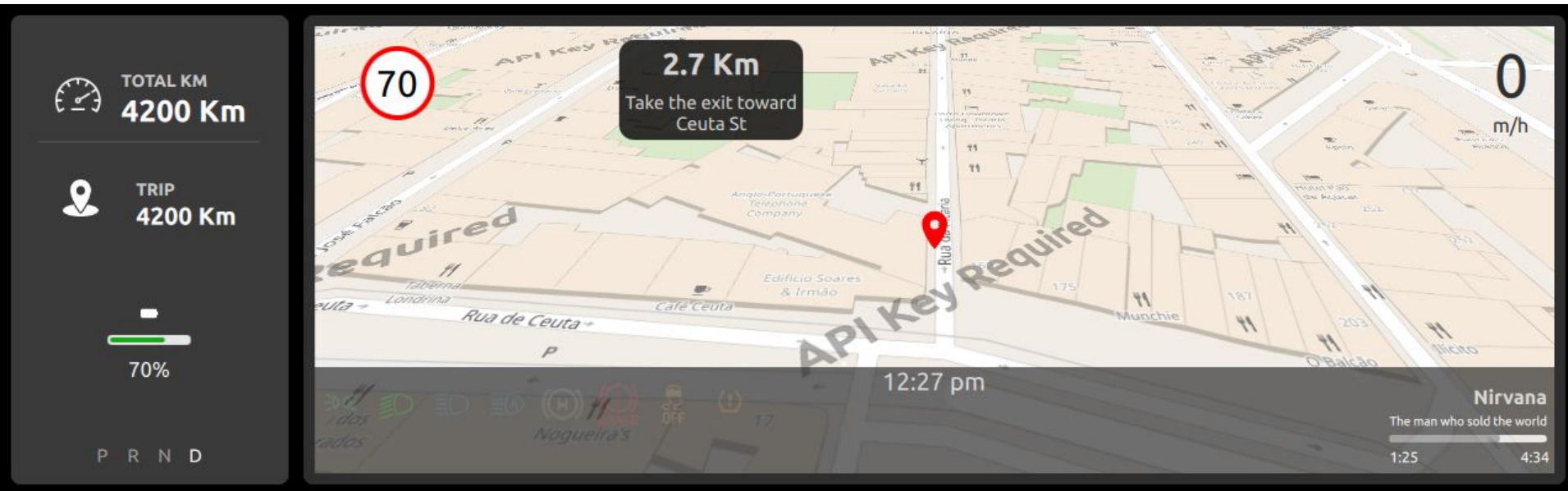
issue :

OFF BOARD CLUSTER



issue :

OFF BOARD CLUSTER



LCOV - code coverage report

Current view: [top level](#)Test: [filtered_coverage.info](#)

Date: 2025-02-11 22:56:43

	Hit	Total	Coverage
Lines:	113	122	92.6 %
Functions:	29	30	96.7 %

Directory	Line Coverage ↕	Functions ↕
Digital_Cluster	92.6 %113 / 122	96.7 %29 / 30

Generated by: [LCOV version 1.14](#)

LCOV - code coverage report

Current view: [top level](#) - [Digital_Cluster](#)Test: [filtered_coverage.info](#)

Date: 2025-02-11 22:56:43

	Hit	Total	Coverage
Lines:	113	122	92.6 %
Functions:	29	30	96.7 %

Filename	Line Coverage ↕	Functions ↕
System.cpp	100.0 %66 / 66	100.0 %17 / 17
zmqreader.cpp	83.9 %47 / 56	92.3 %12 / 13

Generated by: [LCOV version 1.14](#)

CATCH2
tested !!

```
tests system
  • tests system
tests zmqreader
  • tests zmqreader
tests qml
  • tests qml
```

Results

Duration	9.0 sec
Tests	3
Failures	0

tests system

Test case:	tests_system
Outcome:	Passed
Duration:	0.023 sec

Stdout

```
Randomness seeded to: 49068963
All tests passed (8 assertions in 1 test case)
```

tests_zmqreader

Test case:	tests_zmqreader
Outcome:	Passed
Duration:	9.099 sec

Stdout

```
Randomness seeded to: 154690727
=====
All tests passed (2030 assertions in 1 test case)
```

hough transform

The Hough transform is a feature extraction technique used in image analysis, computer vision, pattern recognition, and digital image processing.

canny algorithm

The Canny edge detector is an edge detection operator that uses a multi-stage algorithm to detect a wide range of edges in images. Canny also produced a computational theory of edge detection explaining why the technique works.



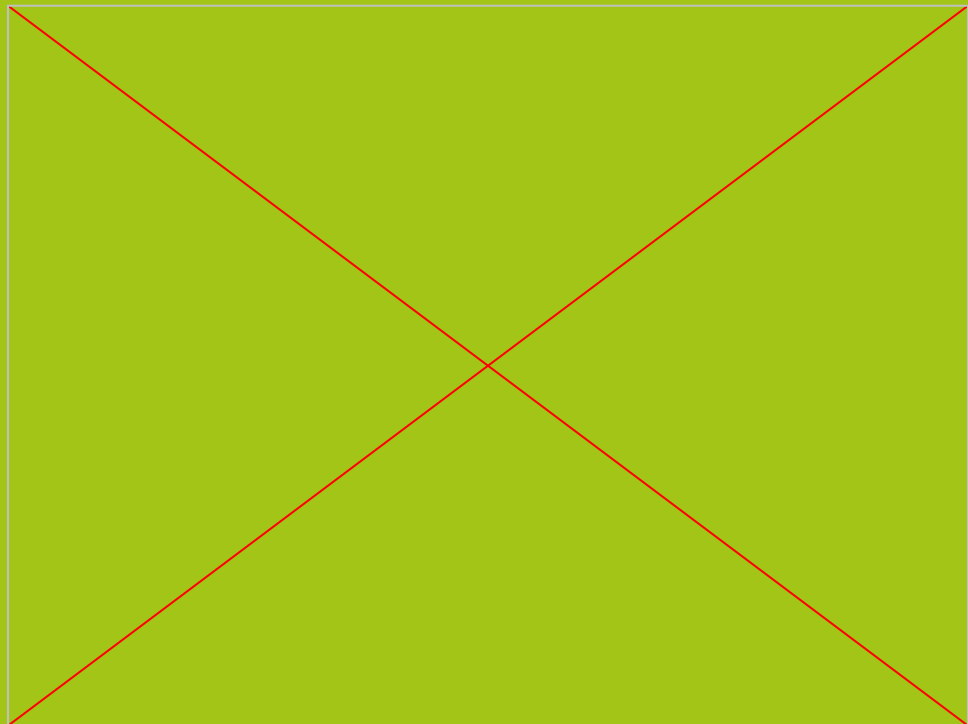
Lane Detection and Steering Angle Estimation

Overview

This project is designed to detect lane markings in a video stream, calculate the curvature of the lanes, estimate the vehicle's offset from the center, and determine the steering angle required for proper alignment.

Utilizing computer vision techniques provided by OpenCV and numerical operations via NumPy, the code processes each video frame through several stages to output a visual overlay of detected lanes with critical driving metrics.

1. Video Capture and Preprocessing
2. Trackbars for Dynamic Thresholding
3. Perspective Transformation and Data Augmentation
4. Thresholding and Mask Generation\n- HSV
5. Lane Detection with Sliding Windows
6. Polynomial Fitting and Lane Parameter Calculation



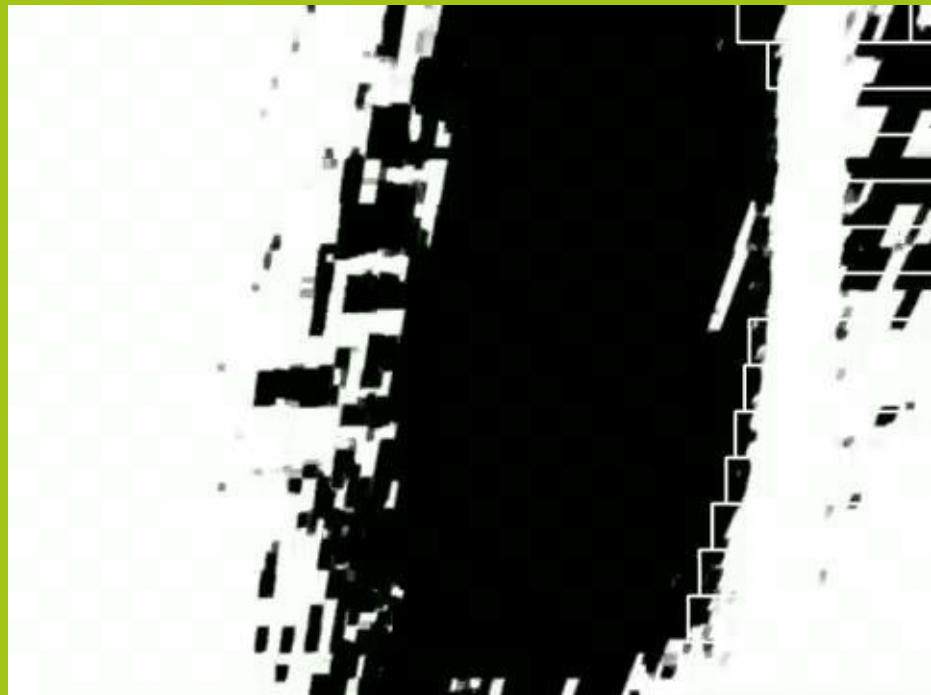
Lane Detection and Steering Angle Estimation

Overview

This project is designed to detect lane markings in a video stream, calculate the curvature of the lanes, estimate the vehicle's offset from the center, and determine the steering angle required for proper alignment.

Utilizing computer vision techniques provided by OpenCV and numerical operations via NumPy, the code processes each video frame through several stages to output a visual overlay of detected lanes with critical driving metrics.

1. Video Capture and Preprocessing
2. Trackbars for Dynamic Thresholding
3. Perspective Transformation and Data Augmentation
4. Thresholding and Mask Generation\n- HSV
5. Lane Detection with Sliding Windows
6. Polynomial Fitting and Lane Parameter Calculation



lane detection

What didn't go well

JetRacer full assembly lane detection accuracy

no impediment detected

finish JetRacer full assembly

improve lane detection accuracy

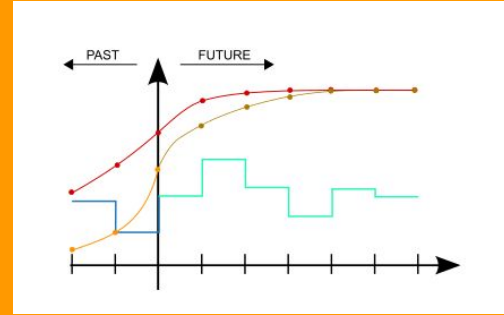
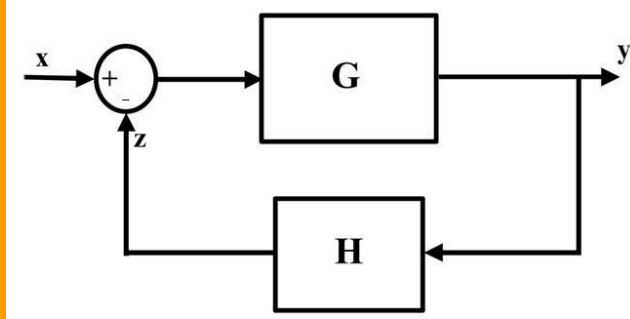
lane detection

SPRINT 3 : planming

build lane detection model

develop the predictive algorithm

classic control (pid) + predictive control



sea!me
software engineering in automotive
and mobility ecosystems

team 07